

LHSC 골밀도 측정 보조 시스템 통합 프로젝트 요구사항 정의서 (PRD)

기관: London Health Sciences Centre (LHSC)

작성자: Senior Software Architect & AI Researcher Edward Seongmin Choi

날짜: 2026년 6월 25일

버전: v1.0

1. 프로젝트 개요

본 프로젝트는 LHSC의 임상 워크플로우를 혁신하기 위한 **AI 기반 2D X-ray 골밀도(BMD) 측정 보조 시스템**이다. 의사가 환자의 **2D X-ray DICOM** 파일을 업로드하면 **AI 모델**이 요추 척추체(L1~L5)를 추출하고, 이 중 **L4** 요추의 골밀도를 계산하여 시각화한다. 과거 이력 데이터 누적에 따른 골밀도 변화량 차트와 함께 진단 신뢰성을 위한 **XAI(Grad-CAM 히트맵)** 리포트를 제공한다. 향후 타 프로젝트와의 결합 및 **Firebase** 전환을 고려하여 백엔드(**FastAPI**)와 프론트엔드(**React/Vite**) 모두 철저히 모듈화하여 설계한다.

2. UI/UX 화면 요구사항 정의서

2.1 [화면 01] 환자 리스트 및 검색 화면 (사이드바)

컴포넌트명	기능 기술	비고 / 이벤트
환자 통합 검색바	환자 이름 또는 ID를 입력하여 실시간 필터링 검색 수행	Debounce(300ms) 처리 적용
환자 마스터 리스트	검색된 환자 목록 표시 (이름, ID, 성별/나이, 최근 검사일)	항목 클릭 시 상세 화면 전환

2.2 [화면 02] 환자 상세 및 X-ray 이력 화면 (중앙 상단)

컴포넌트명	기능 기술	비고 / 이벤트
환자 메타 정보 카드	선택된 환자의 기본 인적 사항 및 기왕력 표시	유연한 데이터 구조 설계
DICOM 파일 업로더	2D X-ray DICOM (*.dcm) 파일 드래그 앤 드롭 업로드	업로드 즉시 백엔드 분석 파이프라인 호출
X-ray 검사 이력 그리드	과거 촬영 및 분석 이력을 최신순으로 정렬하여 표시	행 클릭 시 하단 분석 결과 갱신

2.3 [화면 03] AI 분석 및 핵심 진단 결과 화면 (중앙 하단)

컴포넌트명	기능 기술	비고 / 이벤트
이중 이미지 뷰어	좌측 원본 X-ray 이미지와 우측 L1~L5 척추체 세그멘테이션 마스크 오버레이 비교	고유 컬러 코딩 및 마우스 오버 강조
L4 골밀도 지표 패널	L4 요추 타겟 골밀도 값(g/cm²), T-score , Z-score 대형 노출	정상(녹색)/골감소(황색)/골다 공증(적색) 알림
시계열 골밀도 추이 차트	역대 누적 데이터를 활용한 L4 골밀도 및 T-score 변화량 선형 차트	과거 대비 감퇴/개선율(Δ) 자동 계산
XAI (설명 가능한 AI) 모듈	L4 골밀도 산출에 영향을 준 주요 구역을 Grad-CAM 히트맵으로 시각화	의사의 진단 근거 신뢰성 확보

3. 백엔드(Backend) 요구사항 정의서

- 기술 스택: FastAPI, SQLite (SQLAlchemy ORM), PyTorch, pydicom, OpenCV
- 결합도 제어: Controller -> Service -> Repository -> DB 계층 구조 분리. 주피터 노트북 기반 알고리즘 코드는 app/engine/bmd_analyzer.py 모듈로 격리하여 시스템 독립성 확보.
- 주요 **API** 명세:
 - GET /api/v1/patients : 환자 목록 및 검색
 - POST /api/v1/analysis/upload : DICOM 파일 업로드 및 분석 (**L4** 세그멘테이션 및 BMD, Grad-CAM 결과 반환)
 - GET /api/v1/analytics/trends/{patient_id} : 역대 **L4** 골밀도 시계열 추이 데이터 반환

4. 프론트엔드(Frontend) 요구사항 정의서

- 기술 스택: Vite + React (TypeScript), Zustand/Context API, Recharts/Chart.js, Axios
- 모듈화 및 **Firestore** 전환 전략: 데이터 접근 계층을 services/api.ts 내에 추상화하여, 향후 SQLite REST API 호출 구조에서 **Firestore** Client SDK 구조로 전환할 때 상위 UI 컴포넌트 소스 코드의 수정 없이 내부 통신 로직만 원터치로 교체 가능하도록 설계.

5. 개발환경 및 아키텍처 요구사항 정의서

```
lhsc-bmd-project/
├── backend/
│   ├── app/
│   │   ├── config/          # 설정 파일
│   │   └── database/        # SQLite & ORM 설정
```

```

|   |   | engine/           # Jupyter 노트북 이식 AI 모듈
(bmd_analyzer.py)
|   |   | repository/      # DB CRUD (데이터 접근 계층)
|   |   | routers/         # FastAPI 엔드포인트
|   |   |   main.py        # 백엔드 진입점
|   |   | weights/         # AI 모델 가중치 파일 (.pth)
|   |   | Dockerfile
| frontend/
|   |   | src/
|   |   |   components/    # 공통 UI 컴포넌트
|   |   |   features/      # 도메인 기반 모듈 (patients, bmd-viewer)
|   |   |   services/      # API 계층 (추후 Firebase 교체 지점)
|   |   |   main.tsx
|   |   | Dockerfile
| docker-compose.yml

```

6. 클라우드 배포용 도커 인프라 명세서

6.1 백엔드 도커파일 (backend/Dockerfile)

```

FROM python:3.10-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libgl1-mesa-glx libglib2.0-0 && rm -rf
/var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

6.2 프론트엔드 도커파일 (frontend/Dockerfile)

```

FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:stable-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

6.3 통합 빌드 오케스트레이션 (docker-compose.yml)

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend/data:/app/data
      - ./backend/static:/app/static
  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend
```